

Day1- generate test cases with RAG

**RAG Pipeline**
Enterprise v2.0 • Testleaf

DATA PIPELINE

 Convert to JSON

 Embeddings & Store

SEARCH ENGINES

 Query Preprocessing

 Vector Search

 BM25 Search

 Hybrid Search

 Score Fusion

AI PROCESSING

 Summarize & Dedup

 **Prompt & Schema**

SYSTEM CONFIG

 Settings

Enterprise AI Stack
MongoDB Atlas • Mistral AI • Groq

<

RAG Pipeline >

 Prompt & Schema

Configure system prompt templates



 **Generated Test Cases with RAG Pipeline**

Mode: With RAG Model: openai/gpt-oss-20b 50 MongoDB Matches ⚡ 3,692 / 8000 Tok (46% Budget Safe)

 Export CSV

 **Generated Test Cases (3)**

 Retrieved MongoDB Context (9)

 Linked User Stories (10)

ID	Title	Module	Priority	Type	Preconditions	Test Steps	Expected Results
TC_6355	Positive Verification and Approval of a Pending Prescription	Prescription Management	P1	Functional	1. Doctor user is logged in with valid credentials. 2. Patient has at least one pending prescription in the system. 3. Pharmacy dispensing queue is accessible.	1. Launch the HMS application and navigate to the Prescription module. 2. Select the target patient using UHID. 3. Verify that all pending prescriptions are listed in the 'Pending Verification' grid. 4. Click on the first prescription to open the detail view. 5. Confirm that medication name, dosage, frequency, and route match the physician's order. 6. Click the 'Approve' button and confirm the action. 7. Observe the system's confirmation dialog and wait for the sync process to complete.	1. Pending prescriptions are displayed without errors. 2. Prescription details are accurate and fully populated. 3. Approval triggers a success message: "Prescription approved and queued for dispensing." 4. The prescription record status changes to 'Approved' in the database. 5. The pharmacy module shows the prescription in the dispensing queue within 30 seconds.
TC_6356	Negative Verification of Duplicate Medication in Prescription	Prescription Management	P1	Functional	1. Doctor user is logged in. 2. Patient has a prescription containing two entries of the same medication (e.g., Amoxicillin 500mg BID and Amoxicillin 500ma BID).	1. Open the Prescription module and select the patient. 2. Locate the prescription with duplicate entries. 3. Click to open the prescription detail view. 4. Observe the system's visual cue (red border and warning icon) next to the duplicate drug. 5. Click the 'AI Recommendation' button to view suggested corrections. 6. Modify the dosage of one entry to resolve the duplicate.	1. Duplicate drug is highlighted with a warning icon. 2. AI recommendation panel displays at least one actionable suggestion. 3. After modification, the warning disappears and the prescription is considered valid. 4. System logs the duplicate detection event and the doctor's corrective action in the

rag-mongo-demo-jiraRAG

.env

JS fetch-jira-stories.js U

RAG Pipeline | Testleaf Enterprise X

jira_rag_integration_0743e9bc.plan.md U

Day2-fetch Jira US

← → ↺ ☆ http://localhost:3000



RAG Pipeline
Enterprise v2.0 • Testleaf

DATA PIPELINE

 Convert to JSON

 **Jira to Test Cases**

 Embeddings & Store

SEARCH ENGINES

 Query Preprocessing

 Vector Search

 BM25 Search

 Hybrid Search

 Score Fusion

AI PROCESSING

 Summarize & Dedup

 Prompt & Schema

SYSTEM CONFIG

 Settings

 **Enterprise AI Stack**
MongoDB Atlas • Mistral AI • Groq

< RAG Pipeline >  **Jira to Test Cases** 

Fetch a Jira story, ingest it, and generate test cases

Fetch a user story by issue key, ingest it into MongoDB, retrieve related context, and generate test cases.

✓

Fetch Story

2

Review

3

Ingest to MongoDB

4

Generate Test Cases

1. Fetch Jira User Story

Issue Key

SAM1-8

 **Fetch Story**

 **Ingest into MongoDB**

 **Generate Test Cases**

Configure JIRA_BASE_URL, JIRA_EMAIL, and JIRA_API_TOKEN in Settings before fetching.

SAM1-8 Develop Transaction History Feature [Open in Jira](#) 

Fetches Jira user story

Status: To Do

Priority: None

Module: (Example) Billing System Dev

Task

DESCRIPTION

The Transaction History feature allows users to view their past transactions. This task involves:

Designing the transaction history interface.

Implementing backend logic to retrieve transaction data.

Ensuring data accuracy and security.

This feature is vital for users to track their investment performance over time.

ACCEPTANCE CRITERIA

Not specified in Jira

go-demo-jiraRAG 0 0 0

Cursor Tab → Tab Stats: 4/11 (36%) Agent

fetch_jira_user_stories RagTest_cases +

Cluster0 > RagTest_cases > fetch_jira_user_stories

Visualize Your Data

Documents 2 Aggregations Schema Indexes 1 Validation

🕒 ▼ Type a query: { field: 'value' } or [Generate query](#) 🚀

Options ▶

 EXPORT CODE

+		
---	--	--

▼ Show 7 more fields

©2026 MongoDB, Inc. [Status](#) [Terms](#) [Privacy](#) [Atlas Blog](#) [Contact Sales](#)



RAG Pipeline > Jira to Test Cases

Fetch a Jira story, ingest it, and generate test cases

Fetch Story

Review

Ingest to MongoDB

Generate Test Cases

1. Fetch Jira User Story

Issue Key
SAM1-8

Fetch Story

Ingest into MongoDB

Generate Test Cases

Configure JIRA_BASE_URL, JIRA_EMAIL, and JIRA_API_TOKEN in Settings before fetching.

SAM1-8 Develop Transaction History Feature

Open in Jira

Fetch Jira user story

Status: To Do

Priority: None

Module: (Example) Billing System Dev

Task

DESCRIPTION

The Transaction History feature allows users to view their past transactions. This task involves:

Designing the transaction history interface.

Implementing backend logic to retrieve transaction data.

Ensuring data accuracy and security.

This feature is vital for users to track their investment performance over time.

ACCEPTANCE CRITERIA

Not specified in Jira

Ingested SAM1-8 into fetch_jira_user_stories (new document). Embedding: mistral / mistral-embed (1024 dims).

Generated 5 test cases from SAM1-8

✓ Generated 5 test cases from SAM1-8

DATA PIPELINE

Convert to JSON

Jira to Test Cases

Embeddings & Store

SEARCH ENGINES

Query Preprocessing

Vector Search

BM25 Search

Hybrid Search

Score Fusion

AI PROCESSING

Summarize & Dedup

Prompt & Schema

SYSTEM CONFIG

Settings

Enterprise AI Stack
MongoDB Atlas • Mistral AI • Groq

RAG Pipeline > Jira to Test Cases

Fetch a Jira story, ingest it, and generate test cases

TC generated Successfully..

Generated Results

📘 Coverage from MongoDB: 8 related test cases. Gaps: No test cases verify retrieval of patient-specific transaction history by UHID/PRN; Absence of pagination, date-range filtering and sorting validation; Missing security tests for HIPAA-compliant encryption and role-based access control; No boundary tests for extremely large transaction volumes or zero-transaction scenarios; Lack of audit-trail verification when a user views or exports transaction data

Generated Test Cases (5)

Retrieved MongoDB Context (8)

Similar Stories (2)

ID	Title	Module	Priority	Type	Preconditions	Test Steps
TC_6357	Boundary: Transaction History Pagination with Maximum Page Size	Billing System	P2	Functional	User logged in as Billing Clerk; patient UHID 789012 has 350 transactions spanning the last 5 years; system configuration sets MaxPageSize = 100.	1. Open Billing → Transactions 2. Enter UHID "789012" to cover all transactions 3. Set Page Size to 100 4. Click "Search" 5. Verify that 100 transactions are displayed on Page 1. 6. Click "Next" to verify rows 101-200 are displayed on Page 2. 7. Click "Last" to verify rows 301-350 are displayed on Page 4. 8. Confirm total matches 350.
TC_6358	Security: Role-Based Access Control – Unauthorized User Attempts to View Transaction History	Billing System	P1	Integration	User logged in as Lab Technician with role "LabOperator"; audit logging enabled.	1. From the main menu, navigate to Billing → Transactions 2. Observe UI for role-based restrictions 3. Directly enter https://hms.example.com/billing/transactions in browser 4. Attempt to submit request 5. Capture HTTP response and status code 6. Review audit logs for unauthorized access attempt